Group Report on the

CS7CS3 Advanced Software Engineering Group Project

Group: 3
Daoqi Xiong (25335365), Jiayun Zhang (25334655), Joshua O'Donnell (25353241), Keshav Tripathi (25347551), Priyansh Nayak (25350660), Rundong Chen (25336554), Viraj Bhor (25358463), Xu Gong (25334562), Yokesh Kathiravan (25341409)

This report is intended for you to provide a reflection on the use of eXtreme Programming in your group project.

Instructions:

1. Please give a **"real" assessment of the actual benefits and drawbacks of using XP for your project** under each of the 12 core practices of XP. Indicate when each category was not applicable, and why[1].
2. Change the <group number> in the heading above to your group number
3. Change the <list of group members> to be your group members
4. Change the name of this file to replace the 'N' in the current 'GroupN' with your group number
5. Submit on Blackboard – note ONE SUBMISSION is sufficient for your whole group.

1. Whole Team

"All the contributors to an XP project – developers, business analysts, testers, etc. – work together in an open space, members of one team…."

How was this applied in your team?

As a group, we met at the student lab spaces at Phoenix House during the assigned time slots specified on the timetable submitted to Prof. Clarke, within speaking distance of one another, and coordinating as a whole team on weekly meetings Thursday afternoons.

Benefits as experienced in your team

- When anything wasn't directly explainable in a team or for cross-component feature coordination, the other team members were always available to explain or help out.
- With the project's progress, we developed a fairly good climate within the team from working together.

[1] It is highly unlikely (I would suggest not possible) that any Generative AI tool will be able to tell you what your experiences were in your group. I say this to indicate that a GenAI response to these questions will be treated with considerable skepticism on my part.

- Productivity remained high throughout the duration of the project.

Drawbacks as experienced in your team

- When there were tensions it became awkward very quickly, especially when left to simmer before expression.
- A lack of hierarchy within the group led to inertia in short-term decision-making at times.

Lessons learned

This way of working as a team works well for building a decent work climate both in terms of productivity and interpersonal relations, however, we feel like it would benefit from an integrated outlet for expressing frustrations and issues in a better, more sustainable fashion, e.g. a 'middle manager' figure in charge of making decisions and resolving conflicts or tensions within the team when the consensus or group doesn't step in by itself.

2. Planning Game

"Planning is continuous and progressive. Every two weeks, for the next two weeks, developers estimate the cost of candidate features, and customers select those features to be implemented based upon cost and business value"

How was this applied in your team?

Iteration features were assigned based on discussions within the group on the priority of implementation, informed by regular progress reports both in the formal end of week meetings as well as daily in a dedicated discord text channel for self-reporting the tasks done on the day.

Benefits as experienced in your team

- The structured discussions provided an opportunity for everyone to contribute towards directing the project, alongside a clear division of responsibilities for any given iteration.
- The planning task performed as a group allowed us to concentrate on addressing features based on importance towards the overall project.

Drawbacks as experienced in your team

We found that our meeting pace, set to take place every Thursday at 17:00 should have been better coordinated with iteration progress, given some instances where these decision-making meetings took place either shortly before an iteration was ready to close, or paced a few days after an iteration was ready to formally close. The latter case was less problematic, since it allowed for early integration or extra refactoring to take place, but still could be considered wasted time for some.

Lessons learned

The lesson learnt from here mostly mirrors the factor outlined in the drawbacks, but we found the system to be otherwise well-suited for our project and general satisfaction throughout the group.

3. Customer Tests

"As part of selecting each desired feature, the customers define automated acceptance tests to show that this feature is working"

NOTE – This principle cannot be applied in your team, as you do not have a customer on your team. If you have an opinion on whether this would have been beneficial, please add it here.

Customer tests would have been very helpful towards the end in helping to assess the usability (general controls, feature progression flows, …) of our game, since it is a very big topic especially in VR, but also any desktop apps. This was instead addressed by giving away entire iterations to testing and refining features, most prominently the control mechanisms for VR and desktop users, or UI progression flows.

4. Simple Design

"The team keeps the design exactly suited for the current functionality of the system. It passes all the tests, contains no duplication, expresses everything the authors want expressed, and contains as little code as possible".

How was this applied in your team?

When beginning a new iteration, teams took some time to conceive their feature before going in to implement it, based on questions along the lines of "Is this idea usable/scalable/performant enough? Can it be improved upon?"
Furthermore, we naturally defaulted to improving on existing systems rather than conceiving new items for any given component. Good examples for this are our backend service, which has grown to house the authentication, caching and moderation systems all in one (with due diligence in keeping the various features well compartmentalized), or the book system which features a web of sub-systems ranging from loading and rendering to book management or interaction.

Benefits as experienced in your team

The clear benefit of this was the existence of a performant and well-extensible system, that was robust to dependency on other major components, leading to a cohesive and sturdy project in practise.

Drawbacks as experienced in your team

The main drawback that comes with these systems built over several iterations were the overhead in introducing people to a given system later on, requiring team leads to have a very good overview of a system built by people that have long since moved on to other components, as well as the partial loss of overview on the more intraconnected systems (e.g. the networking component towards the late development stage).

Lessons learned

The system we have implemented has shown to be well executed for scalability, however the extension of systems should have been limited to avoid convoluting systems that grow to much overhead.

5. Pair Programming

"All production software is built by two programmers, sitting side by side, at the same machine".

How was this applied in your team?

For every iteration, we assigned work into four pairs of workers alongside a 'wildcard' member to compensate for the uneven number of group members.
Teams assigned to continue building on a previous iteration were formed by having the former 'associate' graduate to team lead, onboarding someone who hadn't worked on that component before.
In other iterations starting a new feature entirely, the assigned pair determined the team lead based on whoever was willing to take on the 'responsibility' for the iteration.
The wildcard on the other hand was assigned to help out wherever an extra head could come in useful, which turned out to be mostly in helping to playtest ongoing features.

Benefits as experienced in your team

When setting up a table on who worked on what component, it is fairly impressive to see everyone's portfolio being fairly diverse, with some members working on the authentication, caching, book cataloguing, and controls polishing interactions for example.
This helped everyone grow relatively familiar with the various components throughout the project on a personal level.
Also this allowed everyone to contribute their personal areas of expertise to different components, which greatly diversified the project in our eyes.

Drawbacks as experienced in your team

The difficulties most observed came with iterations that required extra overhead in the onboarding process. Some more complex features such as the networking or backend services required some extra work onboarding new people (networking for the sheer complexity of the task, the backend for its relative decoupling from the rest of the project + state as sole component implemented in Java rather than Unity C#).

Larger features also required parallel work that was treated by separate teams during the same iteration, which were fairly difficult to coordinate and merge (notably prompting merge conflicts that took extra work to resolve, or parallel changes on separate pieces that unwittingly broke the other group's functionality).
Also, sometimes the grouping logic had to be amended to account for who would be paired with whom when there was a bit of tension.

Lessons learned

Overall, we feel like the pair programming exercise turned out to be a great contribution towards the project despite the sensibilities involved in planning it.

6. Test-Driven Development

"The programmers work in very short cycles, adding a failing test, then making it work"

How was this applied in your team?

Tests were designed using integrated test runners such as JUnit in the backend systems, or the Unity integrated Test Runner. Members were specifically instructed to start the iteration by planning the feature through the lens of tests to execute for the feature to work as intended.
Towards the end of the development phase, we also started including more and more structured play tests to account for the many complex interactions between systems, in many cases also driven by user input. A good example for this was assessing the usability of our player controls.

Benefits as experienced in your team

The test-driven approach to the task helped structure component features and development priorities along a feature that acted as a checklist for future features, while keeping reminders from past iterations.

Drawbacks as experienced in your team

The many advantages of TDD however translate poorly to the very complex world of gamified systems due to the sheer amount and complexity of displayed interactions.

We found it particularly difficult to implement scripted tests for networked interactions (how do you check whether the texture used on a book is exactly the same on the other player's end), compounded by the fact that observed behaviour in the few networked components that were testable differed from the exact same behaviour run through the integrated play mode test runner, leading to several tests that passed the technical tests but failed the personal 'interview' in subsequent playtests.

Similarly in integrating parallel contributions to a large system, the same difficulties in coordination came through for features that were conventionally testable, which led to some extra effort post-integration. And finally, there were subjective features such as user control usability that simply couldn't be accounted for through scripted tests. For example, when polishing the desktop controls, there was significant effort poured into cleaning up the transitions from various types of user interfaces and the 'default' state of navigating the world freely.

Lessons learned

TDD in our experience provides a good basis for features working on a local level, but is difficult to maintain at scale and fails to generalize beyond local functionality in our scenario of a medium-scale multiplayer video game.

7. Design Improvement

"Don't let the sun set on bad code. Keep the code as clean and expressive as possible"

How was this applied in your team?

Within the iteration flow, we kept accounting for time after formally completing an iteration functionally for cleaning up any code the pair edited over the iteration, checking for readability, potential for optimization or structural improvement and other coding good practises.
Also in interactions building on previous iterations, as mentioned in section 5, we often ended up building on existing subsystems, allowing for different members to interpret it in their own way of how to improve it. This has led to direct, original ideas that improved components structurally and/or functionally.
This was also well helped along by the good maintenance of coding standards regarding naming and general practises (as will be further laid out in later sections) throughout the team, most prominently the segregation of components into individual packages to be added to the scene just by dragging and dropping a prefab.

Benefits as experienced in your team

Our group-wide agreed coding standards helped keep the code decently readable by default, which helped reduce the overhead in introducing new people to a component in not needing to also explain the general structure of a script or system.
Overall, we feel like we were able to keep a very good overview of our project structure thanks to its modularity, cleanliness, and compartmentalization.

Drawbacks as experienced in your team

There were small instances where bad habits could creep through when an iteration was closed a bit too early for example, which were however easy to iron out down the line with thorough refactoring.
A larger observed drawback to our approach of system-building mirrors the French saying that too many chefs oversalt the soup, meaning that some systems (most prominently the expansive book system) underwent comprehensive surgery once too often, which led to instances of components being difficult to untangle when debugging issues down the line.

Lessons learned

We believe that overall we didn't do too badly in this regard, however a bit of oversight on structural changes could have helped address the latter issue, if teams were for example required to pitch their proposed iteration plan before proceeding to implementation.

8. Continuous Integration

"The team keeps the system fully integrated at all times"

How was this applied in your team?

Throughout the project, we have maintained an extensive version control pipeline through the Plastic system, which is directly integrated to Unity. This allowed for us to proceed with regular merges when iterations were ready, which usually ended up being once every week and a half on average.

Benefits as experienced in your team

This regular merging allowed every iteration to build on the current state of the system rather than needing to account for dead code that would have been forgotten. Also for the groups that were tasked with testing features, it didn't involve any branch hopping to find up-to-date systems.

Drawbacks as experienced in your team

While we did manage to get a running rhythm going, in the early stages of the project, some features, most prominently the book system, that were deemed not ready took some inertia integrating to the main repository, which probably set development back somewhat.
Also we ran into many issues using Plastic specifically, which whilst directly integrated to the project, has a fairly frustrating way of 'resolving' conflicts automatically by default without an easy way to do so by hand, which led to several instances where branches (including our main branch) had to be retired entirely since any attempt at merging back came with a truckload of issues from broken commits (which admittedly could have been avoided by the industry practise of not committing broken code, but these commits were usually made in exceptional circumstances for hot fixes).
We had started the project using GitHub but were required to migrate to Plastic due to Unity's insistence of full integration with Unity Cloud services, including PlasticVC for the project to benefit from the multiplayer and networked features, so our hands were tied in looking for free and better alternatives.

Lessons learned

Merging flows should have been more formalized in making sure not to push broken commits to important branches, exacerbated by a bad choice of version control that was partly out of our control, and would have cost much more development time to circumvent.

9. Collective Code Ownership

"Any pair of programmers can improve any code at any time"

How was this applied in your team?

The implementation of pair programming practises inherently made code ownership shared between direct and past collaborators to take accountability for and improve upon. This process was greatly helped along by the fact that the other group members were at hand to answer any questions that couldn't be answered internally.
Furthermore the entire project was made available to the entire team through version control, meaning that no component was outright inaccessible without direct handing-

over from previous owners. Also, groups were asked to self-report their progress daily on our discord server, so that in a pinch anyone would at least know who to talk to for questions.

Benefits as experienced in your team

Interoperability, code readability
This system led to very good interoperability, allowing members to jump between completely unrelated components from one iteration to the next. Also the fact that code was written by everyone, for everyone, meant that there was a constant effort in maintaining a good overview of each component's structure and code.

Drawbacks as experienced in your team

As with previous paradigms, the project ran into scaling issues for certain large systems, with difficulty maintaining a good overview of large, convoluted systems worked on by several groups at a time.
Also in terms of temporal accountability, when people were asked about a certain bug or feature that was deemed bad or inefficient, there were instances of "that wasn't me" responses that indicate some level of detachment.

Lessons learned

Overall we believe the project has maintained a very good level of collective code ownership, however it could be improved by breaking down larger systems or more thorough onboarding processes to make sure the next team leads for a feature fully understand the component they will be working on later.

10. Coding Standard

"All the code in the systems looks as if it was written by a single – very competent – individual"

How was this applied in your team?

Our group placed a great emphasis on maintaining a code standard that was manageable and understandable for everyone. As such, we started defining collective practises early on, ranging from the use of Object-oriented programming and certain naming standards throughout the project, to the requirement to package features separately in the project folder, maintaining comprehensive comments and variable names, and managing branches in a responsible manner.

Benefits as experienced in your team

This high level of standards allowed for very good transitions to new features for everyone involved, both in explaining and understanding existing code, and serving as a benchmark for the code to be written during and 'after' an iteration while refactoring.
This also meant the project was very scalable in large part thanks to the compartmentalization of components, which could be integrated into the game scene in a simple action rather than requiring a full integration process spanning hours or days, also helping transferability of simple features that could be bound to other components.

Drawbacks as experienced in your team

In this regard we don't believe to have experienced any drawbacks, the advantages were too good for everyone to disregard this aspect of our project's development process. If anything, we were scared initially to have very differing ideas of these standards, which would make them difficult to enforce, but thankfully they were fairly easy to put in place and make a habit.

Lessons learned

Good code and projects need corresponding care from everyone involved.

11. Metaphor

"The team develops a common vision of how the program works"

How was this applied in your team?

From the start, we sat down to talk extensively about our impressions on, and expectations of the project, allowing us to coalesce our view of the project's end product around a sturdy, shared ground truth. However we didn't formalize it so far as to restrain room for interpretation, which probably reflected negatively on our formal architectures in the first weeks of the project.
These interpretations were then progressively discussed and kept agreeable within the group through our open discussions section at the end of our end of week meetings, where anyone could address any question or topic for discussion to everyone.

Benefits as experienced in your team

The great experienced benefit from this was that, past the formalization of our project's formal requirements and structure, we ended up never doubting our project structure or the general direction it was going in, which really helped harmonize the workflow and ease decisions around priorities for iterations when assigning them.

Drawbacks as experienced in your team

As mentioned in the explanation, this informal agreement translated poorly to our initial formal project descriptions, and initially led to a lot of discussions to make this group-wide agreement clear, however we feel like the latter especially was worth the clarity down the line.

Lessons learned

It's a very good thing to have a common idea of what the project should look like, but it would have been great if we had the early organization to bring that to life in the requirement and architectural specifications.

12. Sustainable Pace

"The team is in it for the long term. They work hard, at a pace that can be sustained indefinitely. They conserve their energy, treating the project as a marathon rather than a sprint"

How was this applied in your team?

We did our best to keep a sustainable pace by keeping to the 10-hour weeks set as an assignment requirement.

Benefits as experienced in your team

The best benefit we experienced was the 'workday mentality' of being able to check out of the project off-hours while push wasn't coming to shove.

Drawbacks as experienced in your team

We believe the project description and complexity was very inappropriate for the expected amount of time given to work on this project. We already took care to reduce the requirements compared to the original proposal (e.g. not considering the integration of an external website for book handling, nor the integration of a subscription service to manage users), yet the project still proved a big mouthful for us to chew collectively, especially paired with the pressure of our relatively bad grades at the early stages of the project.
So, especially towards the close of our project's development phase, we had to trade off between showcasing a good project at the demo, or keeping to the 'sustainable pace' set by the timetable.
This was also in part exacerbated by the scare from Unity deciding to announce the discontinuation of a multiplayer service. While it did end up not affecting us in the end, thankfully, it did prompt an early crunch phase to demonstrate our networked features ahead of time.

Lessons learned

It is unfortunately incredibly common in the gaming industry to have crunch phases around deadlines for releases and showcases, a fact we have in the end learnt the hard way.

13. Overall Project

Benefits as experienced in your team

Throughout the project, we have built a project that can stand on its own two legs, with plenty of experience for showcase on our portfolios going into this uncertain job market. We have naturally evolved functional workflows that we believe will also translate well to the workplace, and skills we have previously not been exposed to, such as handling real-time, networked applications, or handling the complexities of stringing together many components that interact in non-straightforward manner.

Also we believe to form a pretty good group by now, interpersonally speaking, which is always a plus.

Drawbacks as experienced in your team

We worked on a project which was expansive in scale and complexity, both directly for our capabilities and, in our opinion, compared to other projects given to groups in other strands of the Computer Science cohort.
Throughout the project's lifetime we were also constantly held back by needing to work around the unavailability of any real support towards the many issues we encountered in our development process:

- We were completely left to ourselves for network access, needing to resort to mobile hotspots (which are overloaded by even just 4 devices) for even access to version control or basic multiplayer features, because TCDWifi completely locked these features despite back-and-forth with the IT services throughout the entire project.
- We were consistently denied any form of funding or compensation for many features that either came at a premium, or unfeasable development cost to address, including
    - Unity's multiplayer infrastructure,
    - deployment of our backend services to cloud servers and
    - linking them to a registered and accessible domain name, or
    - accessing qualitative 3D assets for a visually enticing environment.

Each of these led to either payments out of our own pocket, or unsavoury workarounds, e.g. the requirement of a hard-coded IP address to access the backend. Each of these deficiencies would have already been straining individually, but ended up compounding to a scary list of issues that affect several core components and requirements of our project.

To summarize, we feel like this was a project that is steep to begin with, where we felt abandoned in addressing issues that were largely outside of our control (particularly the network unavailability was incredibly frustrating and destructive to our project's showcases).

Lessons learned

The benefits of a strong and functional organization are worth the effort, and in the end we have a project we can be proud of to showcase for the future, but it takes investment to flourish, both personal and from outside. At university it's dealing with sysadmins and financial restrictions, in the industry it will be dealing with server hosting and investors, so it's a bittersweet reality-check for projects at this scale.

14. Analysis of the use of Generative AI tools in your project, as experienced in your team

Strengths of Gen AI tools as experienced in your team across the whole development lifecyle (where relevant). Please give explicit examples.

We found generative AI to be particularly useful in accessing up-to-date documentation both for Unity's features, as well as for other external API's, where the scale of these documents (or just bad work on the providers' side :) led to some documentation being deprecated, and alternatives hard to find using regular queries. This was most useful in accessing the Unity networking features, which undergo regular changes from one update to the next, and end up changing faster than they can update their API it seems.

We also found it very useful for debugging unclear error messages especially for more 'black box' features such as the networking and HTTP features.
Finally, we also found that it did augment our capabilities in ideating higher level features, and accessing components and software that would otherwise have taken extensive research and practise to use as we did in this project. For example, the book system features a lot of features that were ideated by generative AI, such as the use of Render Textures to bake a book's text into objects to make them directly editable for our annotations, rather than trying to overlay transparent images over UI text fields, which have looked terrible in practise before the thin slice demo.

GenAI was also very helpful in getting members up to speed on C#, which while structurally fairly similar to Java still would have taken a bit of overhead for some to learn otherwise.

Limitations of Gen AI tools as experienced in your team across the whole development lifecyle (where relevant). Please give explicit examples.

While we mentioned above its ability to augment our capabilities in conceiving more complex systems, we find that it didn't quite translate to implementing them when left too much liberty, where we had several components that took significant effort to untangle after having been left unchecked for too long. The best example for this is, again, the book system, where Claude had been allowed direct access to project files, which worked out fine on the surface for testing.

When this code needed to be reassessed for usability and integration down the line, it came out to a very convoluted mass of code, including dead code that proved a nightmare to debug and clean out.

On a smaller scale, there were instances where debugging with ChatGPT turned out to be cyclical in nature when coming up with solutions to previous breaks. For example when debugging a network issue, it will confidently diagnose fix A. When fix A doesn't work, it will say to use function B instead, which was deprecated since 2024.

After a bit of back and forth it will then have lost context and bring back fix A again, leading to some frustration and a bit of extra work time as a result.

Lessons learned

We have seen GenAI being used to very good effect throughout the project, enabling us to tackle a very complex system successfully in no small part thanks to its augmenting capabilities. However, we did find out the hard way that sometimes, it does need to be steered by a context-aware user to avoid a potential explosion later.

In short, when used responsibly, GenAI has the potential to make a major impact on a project's success.